

3. Exercise: Vertex Cover

Abstract

This study presents two algorithms for solving the Vertex Cover problem. The first is a 2-APX approximation algorithm that provides a factor-2 approximation for the vertex cover. The second is a Fixed-Parameter Tractability (FPT) algorithm employing a Bounded Search Tree (BST). The FPT algorithm explores the existence of a vertex cover with a specified size parameter, offering practical feasibility for large instances. Theoretical explanations, algorithm implementations, and an experimental evaluation showcase the effectiveness of these approaches, shedding light on versatile strategies for addressing the Vertex Cover problem.

1 Introduction

In the landscape of computer science, where computational challenges continually evolve, the Vertex Cover problem stands out as a quintessential conundrum in graph theory. This problem entails identifying the most compact subset of vertices in a given graph, ensuring that each edge connects to at least one vertex within the subset. The inherent difficulty arises from the NP-hard nature of the problem, making exact solutions computationally expensive for larger instances.

This document introduces and explores two algorithmic strategies to address the Vertex Cover problem. Firstly, a 2-APX approximation algorithm is employed to provide a solution within a factor-2 approximation, presenting a trade-off between optimality and computational feasibility. Secondly, a Fixed-Parameter Tractability (FPT) algorithm is introduced, leveraging the concept of a Bounded Search Tree (BST). The FPT algorithm focuses on determining the existence of a vertex cover with a specific size parameter, thereby offering a more nuanced and pa-

rameterized approach to the problem.

The journey unfolds with theoretical foundations, explaining the intricacies of the Vertex Cover problem and introducing the algorithmic paradigms. Subsequently, the algorithms are implemented and subjected to empirical evaluation. The outcomes of these evaluations, coupled with their theoretical underpinnings, shed light on the adaptability and effectiveness of these algorithms in practical scenarios. By exploring these diverse algorithmic strategies, this document aims to contribute to the ongoing discourse on efficient problem-solving approaches in the domain of graph theory.

2 Preliminaries

2.1 Vertex Cover

In the context of a graph $G(V, E)$, the vertex cover means the smallest node subset $C \subseteq V$ that satisfies $\forall e \in E : e \cap C \neq \emptyset$.

2.2 Approximative Algorithm

As the exact solution is NP-hard, we first implement a simple factor-2 approximation. This algorithm picks a random edge and adds both adjacent to the vertex cover. Then all edges adjacent to one of the two vertices are removed from the graph. This is repeated until no edge is left. The vertex cover included all vertices from the edges we chose arbitrarily for each edge. Time complexity: $O(m)$ where m is the number of edges as each edge can be considered only once: either to add its vertices to the vertex cover or to be removed the the graph.

Algorithm 1: 2-APX vertex cover

Data: A graph G with vertices V and edges E

Result: Vertex Cover C with $C \subseteq V$ and

$$\forall e \in E : e \cap C \neq \emptyset$$

```
1  $C \leftarrow \emptyset$ 
2 while  $E \neq \emptyset$  do
3    $e \leftarrow \{v, w\}$  // pick arbitrary edge from  $E$ 
4    $E \leftarrow E \setminus \{x = \{y, z\} | x \in$ 
      $E \wedge \{v, w\} \cap \{x, y\} \neq \emptyset\}$ 
5    $C \leftarrow C \cup \{v, w\}$ 
```

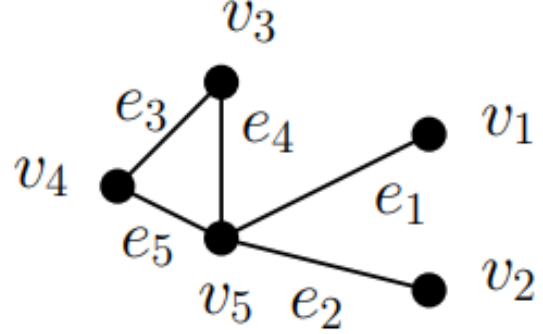


Figure 1: Example input graph

2.3 Parameterized Algorithms

Parameterized algorithms try to capture certain aspects of a problem to find an algorithm that depends on the input size n and some parameter k with a running time of $\mathcal{O}(\text{poly}(n) * f(k))$ meaning polynomial in respect to the input size, but the function $f(k)$ can be an exponential function. n . Therefore, if the parameter k is small enough, the algorithm is feasible for practical use.

Algorithms that are polynomial in the input size but exponential in the parameter value are also called **FPT**(fixed-parameter tractable). Storandt [2023]

3 Algorithm & Implementation

3.1 BST Vertex Cover Algorithm

To make an FPT algorithm for the vertex cover problem we modify the problem to find if a vertex cover of size k is possible or not. We can use it as we can use this problem to find the minimum size vertex cover. The Bounded Search Tree (BST) is an FPT algorithm where we try to solve the vertex cover problem so that its run time is polynomial with respect to the input size and exponential with respect to the parameter k .

The main idea of the algorithm is to start with an edge whose one of the vertices is definitely in the vertex cover. One such type of edge is an edge part of a leaf node. Then we construct a tree-like structure where the nodes of the tree represent the edges of our input graph and the edges of the trees

represent the vertices of the graph we are taking into consideration for our vertex cover. This is shown in Figure 1 and Figure 2 with the help of an example. Also, once we pick an edge in one iteration we remove all the edges linked to the vertices of the picked edges from our input graph. In the next level of the tree, we pick an arbitrary edge to proceed since the algorithm does an exhaustive search so the order we pick the edges in every step does not matter. Also, once the tree reaches a depth of k we check if it is a vertex cover. If it is we return true else we put the edges back to the graph (backtrack 1 step) return false and continue our search with other edges.

The time complexity of this algorithm is $\mathcal{O}(m * 2^k)$, where m is the number of edges and k is the size of vertex cover we need to check for existence. Also, it is of the form $\mathcal{O}(\text{poly}(n) * f(k))$, where $f(k)$ is exponential and $\text{poly}(n)$ is $\mathcal{O}(n^2)$ which is polynomial.

4 Experimental Evaluation

4.1 Data and Hardware

4.1.1 Dataset Generation

The PACE 2019 graphs are used as benchmark data.

Because, we noticed that the benchmark graphs

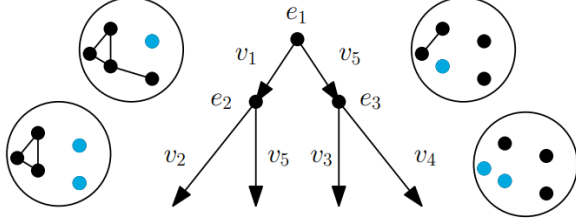


Figure 2: Bounded search tree for the corresponding input graph

Algorithm 2: BST Vertex Cover Algorithm

Data: A graph G with vertices V and edges E

Result: Boolean: If a vertex Cover C with $C \subseteq V$ and $\forall e \in E : e \cap C \neq \emptyset$ of size k exists.

```

1 BST-VC( $G, C, k$ )
  begin
2   if  $|C| = k$  and  $E \neq \emptyset$  then
3     return  $\emptyset$ 
4   if  $E = \emptyset$  then
5     return  $\emptyset$ 
6    $e = \{v, w\} \leftarrow$  pick arbitrary edge from  $E$ 
      $G' \leftarrow G - \{v\}$ 
      $C' \leftarrow C \cup \{v\}$ 
     if  $BST-VC(G', C', k) \neq \emptyset$  then
7     return  $C'$ 
8    $G'' \leftarrow G - \{w\}$ 
      $C'' \leftarrow C \cup \{w\}$ 
     if  $BST-VC(G'', C'', k) \neq \emptyset$  then
9     return  $C''$ 

```

are too large for the BST algorithm, we created some test graphs as well with $10 < n \leq 50$ vertices as well as $1 < m \leq \frac{n * (n - 1)}{3}$ edges.

4.1.2 Hardware

Both the algorithms were evaluated on a machine with an Apple M2 CPU with 3.49 GHz and a memory of 32GB.

4.2 Results

We evaluated the 2-APX as well as BST algorithm both on the generated test graphs as well as the PACE challenge graphs.

For the BST algorithm, we used for the parameter k the values ("5" "10" "20" "50" "100" "200" "400" "800" "1600" "2400" "3600") for the PACE challenge dataset as well as ("5" "10" "15" "20" "25" "30" "35" "40" "45" "50") for the test data. We used a timeout limit of 10 min. In the evaluation, BST is run with the smallest k value. If the BST returns a vertex cover, the instance is considered done. Otherwise the next larger possible k value is chosen until a vertex cover is found or the timeout limit is reached.

Unfortunately, the BST algorithm does not terminate within the time limit for any of the instances. With $k = 50$, in every case, the time limit was reached, what was expectable for the analysed theoretical running time of the algorithm.

For that reason, in the further analysis, we use our generated test data to evaluate the BST algorithm.

The run time results for the 2 approximation algorithm are shown in Figure 3.

Also, the run time results of the BST method are shown in Figure 4. Here plot is between different values of k (till it was feasible) and the average time to run the algorithm for these k in ms.

5 Discussion and Conclusion

In the run time analysis of the 2 approximation method, the time clearly seems to be linearly increasing with the size of the vertex cover.

In the run-time analysis of the BST algorithm, the

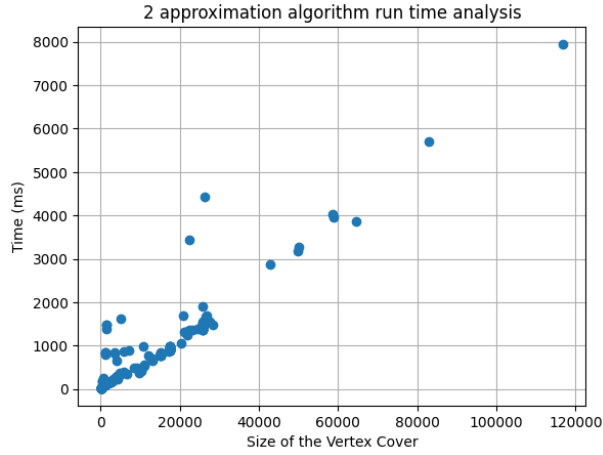


Figure 3: Run time results for the 2 approximation algorithm

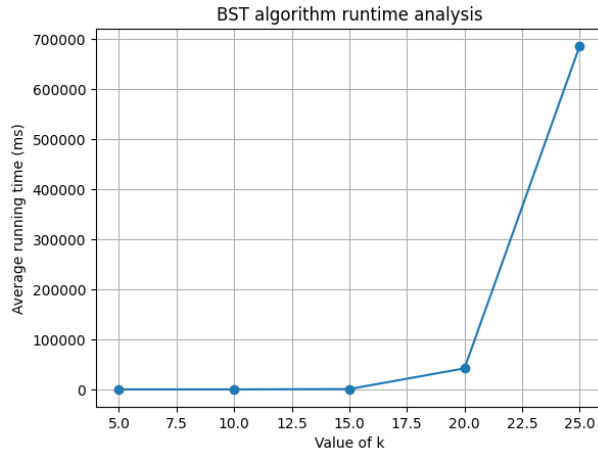


Figure 4: Run time results of the BST method

time is increasing exponentially with respect to out k , which is as per expectation as the time complexity of the algorithm is $\mathcal{O}(m * 2^k)$.

The simple BST algorithm is only feasible for very small values of k and not useable for many graphs found in practice. In contrast to that, the factor-2 approximation is blazingly fast and well suited, if an approximate solution is acceptable.

References

S. Storandt. Lecture: 6. parameterized algorithms - 1, 2023.